VGG16_Quantization_Aware_Training_4bit4bit

December 12, 2025

[ ]:
```
import argparse
import os
import time
import shutil

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import torch.backends.cudnn as cudnn


import torchvision
import torchvision.transforms as transforms

from models import *


global best_prec
use_gpu = torch.cuda.is_available()
print('=> Building model...')



batch_size = 128
model_name = "VGG16_project"
model = VGG16_project()

print(model)

normalize = transforms.Normalize(mean=[0.491, 0.482, 0.447], std=[0.247, 0.243,␣
 ↪0.262])


train_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
```

```
    download=True,
    transform=transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        normalize,
    ]))
trainloader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_size,␣
 ↪shuffle=True, num_workers=2)


test_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
    transform=transforms.Compose([
        transforms.ToTensor(),
        normalize,
    ]))

testloader = torch.utils.data.DataLoader(test_dataset, batch_size=batch_size,␣
 ↪shuffle=False, num_workers=2)


print_freq = 100 # every 100 batches, accuracy printed. Here, each batch␣
 ↪includes "batch_size" data points
# CIFAR10 has 50,000 training data, and 10,000 validation data.

def train(trainloader, model, criterion, optimizer, epoch):
    batch_time = AverageMeter()
    data_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()

    model.train()

    end = time.time()
    for i, (input, target) in enumerate(trainloader):
        # measure data loading time
        data_time.update(time.time() - end)

        input, target = input.cuda(), target.cuda()

        # compute output
        output = model(input)
        loss = criterion(output, target)
```

```
        # measure accuracy and record loss
        prec = accuracy(output, target)[0]
        losses.update(loss.item(), input.size(0))
        top1.update(prec.item(), input.size(0))

        # compute gradient and do SGD step
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # measure elapsed time
        batch_time.update(time.time() - end)
        end = time.time()


        if i % print_freq == 0:
            print('Epoch: [{0}][{1}/{2}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Data {data_time.val:.3f} ({data_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                   epoch, i, len(trainloader), batch_time=batch_time,
                   data_time=data_time, loss=losses, top1=top1))



def validate(val_loader, model, criterion ):
    batch_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()

    # switch to evaluate mode
    model.eval()

    end = time.time()
    with torch.no_grad():
        for i, (input, target) in enumerate(val_loader):

            input, target = input.cuda(), target.cuda()

            # compute output
            output = model(input)
            loss = criterion(output, target)

            # measure accuracy and record loss
            prec = accuracy(output, target)[0]
            losses.update(loss.item(), input.size(0))
```

```
            top1.update(prec.item(), input.size(0))

            # measure elapsed time
            batch_time.update(time.time() - end)
            end = time.time()

            if i % print_freq == 0:  # This line shows how frequently print out the status. e.g., i%5 => every 5 batch, prints out
                print('Test: [{0}/{1}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                   i, len(val_loader), batch_time=batch_time, loss=losses,
                   top1=top1))

    print(' * Prec {top1.avg:.3f}% '.format(top1=top1))
    return top1.avg


def accuracy(output, target, topk=(1,)):
    """Computes the precision@k for the specified values of k"""
    maxk = max(topk)
    batch_size = target.size(0)

    _, pred = output.topk(maxk, 1, True, True)
    pred = pred.t()
    correct = pred.eq(target.view(1, -1).expand_as(pred))

    res = []
    for k in topk:
        correct_k = correct[:k].view(-1).float().sum(0)
        res.append(correct_k.mul_(100.0 / batch_size))
    return res


class AverageMeter(object):
    """Computes and stores the average and current value"""
    def __init__(self):
        self.reset()

    def reset(self):
        self.val = 0
        self.avg = 0
        self.sum = 0
        self.count = 0

    def update(self, val, n=1):
```

```
        self.val = val
        self.sum += val * n
        self.count += n
        self.avg = self.sum / self.count


def save_checkpoint(state, is_best, fdir):
    filepath = os.path.join(fdir, 'checkpoint.pth')
    torch.save(state, filepath)
    if is_best:
        shutil.copyfile(filepath, os.path.join(fdir, 'model_best.pth.tar'))

#model = nn.DataParallel(model).cuda()
#all_params = checkpoint['state_dict']
#model.load_state_dict(all_params, strict=False)
#criterion = nn.CrossEntropyLoss().cuda()
#validate(testloader, model, criterion)
```

[ ]:
```
def adjust_learning_rate(optimizer, epoch):
    """For resnet, the lr starts from 0.1, and is divided by 10 at 80 and 120␣
 ↪epochs"""
    adjust_list = [60, 80]
    if epoch in adjust_list:
        for param_group in optimizer.param_groups:
            param_group['lr'] = param_group['lr'] * 0.1
```

[ ]:
```
# This cell won't be given, but students will complete the training

lr = 4e-2
weight_decay = 1e-4
epochs = 100
best_prec = 0

#model = nn.DataParallel(model).cuda()
model.cuda()
criterion = nn.CrossEntropyLoss().cuda()
optimizer = torch.optim.SGD(model.parameters(), lr=lr, momentum=0.9,␣
 ↪weight_decay=weight_decay)
#cudnn.benchmark = True

if not os.path.exists('result'):
    os.makedirs('result')
fdir = 'result/'+str(model_name)
if not os.path.exists(fdir):
    os.makedirs(fdir)
```

```
for epoch in range(0, epochs):
    adjust_learning_rate(optimizer, epoch)

    train(trainloader, model, criterion, optimizer, epoch)

    # evaluate on test set
    print("Validation starts")
    prec = validate(testloader, model, criterion)

    # remember best precision and save checkpoint
    is_best = prec > best_prec
    best_prec = max(prec,best_prec)
    print('best acc: {:1f}'.format(best_prec))
    save_checkpoint({
        'epoch': epoch + 1,
        'state_dict': model.state_dict(),
        'best_prec': best_prec,
        'optimizer': optimizer.state_dict(),
    }, is_best, fdir)
```

[18]:
```
PATH = "./result/VGG16_quant4b4b/model_best.pth.tar"
model_name = "VGG16_project"
model = VGG16_project()
checkpoint = torch.load(PATH)
model.load_state_dict(checkpoint['state_dict'])
device = torch.device("cuda")

model.cuda()
model.eval()

test_loss = 0
correct = 0

with torch.no_grad():
    for data, target in testloader:
        data, target = data.to(device), target.to(device) # loading to GPU
        output = model(data)
        pred = output.argmax(dim=1, keepdim=True)
        correct += pred.eq(target.view_as(pred)).sum().item()

test_loss /= len(testloader.dataset)

print('\nTest set: Accuracy: {}/{} ({:.0f}%)\n'.format(
        correct, len(testloader.dataset),
        100. * correct / len(testloader.dataset)))
```

```
Test set: Accuracy: 9112/10000 (91%)
```

[ ]:
```
class SaveOutput:
    def __init__(self):
        self.outputs = []
    def __call__(self, module, module_in):
        self.outputs.append(module_in)
    def clear(self):
        self.outputs = []

######### Save inputs from selected layer ##########
save_output = SaveOutput()
i = 0
counter = 0
for layer in model.modules():
    i+=1
    if isinstance(layer, QuantConv2d):
        print(i,"-th layer prehooked")
        counter+=1
        print(layer, counter)
        layer.register_forward_pre_hook(save_output)
####################################################

dataiter = iter(testloader)
images, labels = next(dataiter)
images = images.to(device)
out = model(images)
```

[20]:
```
w_bit = 4
weight_q = model.features[27].weight_q # quantized value is stored during the␣
 ↪training
w_alpha = model.features[27].weight_quant.wgt_alpha   # alpha is defined in␣
 ↪your model already. bring it out here
w_delta = w_alpha/(2**(w_bit-1)-1)   # delta can be calculated by using alpha␣
 ↪and w_bit
weight_int = weight_q/w_delta # w_int can be calculated by weight_q and w_delta
# print(weight_int) # you should see clean integer numbers
print(f"Weight Int Shape: {weight_int.shape}")
```

```
Weight Int Shape: torch.Size([8, 8, 3, 3])
```

[21]:
```
x_bit = 4
x = save_output.outputs[8][0]  # input of the 2nd conv layer
print(f"Input Shape: {x.shape}")
x_alpha  = model.features[27].act_alpha
x_delta = x_alpha/(2**x_bit-1)

act_quant_fn = act_quantization(x_bit) # define the quantization function
```

```
x_q = act_quant_fn(x, x_alpha)          # create the quantized value for x

x_int = x_q/x_delta
print(f"Input Int Sample: {x_int[0,0,0,:5]}")
```

```
Input Shape: torch.Size([128, 8, 4, 4])
Input Int Sample: tensor([3.0000, 0.0000, 0.0000, 0.0000], device='cuda:0',
       grad_fn=<SelectBackward0>)
```

[22]:
```
#### input floating number / weight quantized version

conv_ref = torch.nn.Conv2d(in_channels = 8, out_channels=8, kernel_size = 3,␣
 ↪padding=1, bias = False)
conv_ref.weight = torch.nn.parameter.Parameter(weight_int)
output_int = conv_ref(x_int)

output_recovered = output_int*x_delta*w_delta
relu = nn.ReLU(inplace=True)
relu_output_recovered = relu(output_recovered)

output_ref = save_output.outputs[9][0]
```

[23]:
```
difference = abs( output_ref - relu_output_recovered )
# difference = abs( output_ref - output_recovered )
print(difference.mean())  ## It should be small, e.g.,2.3 in my trainned model
```

```
tensor(2.7784e-07, device='cuda:0', grad_fn=<MeanBackward0>)
```

[32]:
```
x_pad = torch.zeros(128, 8, 6, 6).to(x_int.device)
x_pad[:, :, 1:5, 1:5] = x_int
X = x_pad[0]
x_pad_in=X
X = X.reshape(X.size(0), -1)
X.size()
```

[32]:
```
torch.Size([8, 36])
```

[25]:
```
tile_id = 0
nij = 200 # just a random number
#X = a_tile[tile_id,:,nij:nij+64]  # [tile_num, array row num, time_steps]

bit_precision = 8
file = open('activation4b4b.txt', 'w') #write to file
file.write('#time0row7[msb-lsb],time0row6[msb-lst],....,time0row0[msb-lst]#\n')
file.write('#time1row7[msb-lsb],time1row6[msb-lst],....,time1row0[msb-lst]#\n')
# file.write('#................#\n')

for i in range(X.size(1)):  # time step
```

```
    for j in range(X.size(0)): # row #
        X_bin = '{0:08b}'.format(int(X[7-j,i].item()+0.001))
        for k in range(bit_precision):
            file.write(X_bin[k])
        #file.write(' ')  # for visibility with blank between words, you can use
    file.write('\n')
file.close() #close file
```

[26]:
```
print(weight_int.size())
W = torch.reshape(weight_int, (weight_int.size(0), weight_int.size(1), -1))
print(W.size())
```

```
torch.Size([8, 8, 3, 3])
torch.Size([8, 8, 9])
```

[27]:
```
bit_precision = 8
file = open('weight4b4b.txt', 'w') #write to file
file.write('#col0row7[msb-lsb],col0row6[msb-lst],....,col0row0[msb-lst]#\n')
file.write('#col1row7[msb-lsb],col1row6[msb-lst],....,col1row0[msb-lst]#\n')
# file.write('#................#\n')
for kij in range(9):
    for i in range(W.size(0)):
        for j in range(W.size(1)):
            if (W[i, 7-j, kij].item()<0):
                W_bin = '{0:08b}'.format(int(W[i,7-j,kij].
 ↪item()+2**bit_precision+0.001))
            else:
                W_bin = '{0:08b}'.format(int(W[i,7-j,kij].item()+0.001))
            for k in range(bit_precision):
                file.write(W_bin[k])
            #file.write(' ')  # for visibility with blank between words, you␣
 ↪can use
        file.write('\n')
file.close() #close file
```

[28]:
```
W[0,:,0]
```

[28]:
```
tensor([-5.0000, -0.0000,  2.0000,  7.0000,  7.0000,  3.0000, -3.0000,  2.0000],
       device='cuda:0', grad_fn=<SelectBackward0>)
```

[29]:
```
p_nijg = range(X.size(1)) ## psum nij group
psum = torch.zeros(8, len(p_nijg), 9).cuda()
for kij in range(9):
    for nij in p_nijg:       # time domain, sequentially given input
        m = nn.Linear(8, 8, bias=False)
        m.weight = torch.nn.Parameter(W[:,:,kij])
        psum[:, nij, kij] = m(X[:,nij]).cuda()
bit_precision = 16
```

```
file = open('psum.txt', 'w') #write to file
file.write('#time0col7[msb-lsb],time0col6[msb-lst],....,time0col0[msb-lst]#\n')
file.write('#time1col7[msb-lsb],time1col6[msb-lst],....,time1col0[msb-lst]#\n')
file.write('#................#\n')
for kij in range(9):
    for i in range(psum.size(1)):
        for j in range(psum.size(0)):
            if (psum[7-j,i,kij].item()<0):
                P_bin = '{0:016b}'.format(int(psum[7-j,i,kij].
 ↪item()+2**bit_precision+0.001))
            else:
                P_bin = '{0:016b}'.format(int(psum[7-j,i,kij].item()+0.001))
            for k in range(bit_precision):
                file.write(P_bin[k])
            #file.write(' ')  # for visibility with blank between words, you␣
 ↪can use
        file.write('\n')
file.close()
```

[30]:
```
# out = relu(output_int[0])
out = output_int[0]
out = torch.reshape(out, (out.size(0), -1))
bit_precision = 16
file = open('output4b4b.txt', 'w') #write to file
file.write('#time0col7[msb-lsb],time0col6[msb-lst],....,time0col0[msb-lst]#\n')
file.write('#time1col7[msb-lsb],time1col6[msb-lst],....,time1col0[msb-lst]#\n')
# file.write('#................#\n')

for i in range(out.size(1)):
    for j in range(out.size(0)):
        if (out[7-j,i].item()<0):
            O_bin = '{0:016b}'.format(int(out[7-j,i].item()+2**bit_precision+0.
 ↪001))
        else:
            O_bin = '{0:016b}'.format(int(out[7-j,i].item()+0.001))
        for k in range(bit_precision):
            file.write(O_bin[k])
        #file.write(' ')  # for visibility with blank between words, you can use
    file.write('\n')
file.close()
```

[ ]:
```
# Modified code to show MAC array output (8x16) for each cycle in hex format
import torch

def generate_debug_data_4b4b_with_mac_output(x_pad_in, weight_int):
    """
    Generate debug data for 4b4b systolic array simulation with MAC array output
```

```
    Args:
        x_pad_in: Input activations with padding [8, 6, 6]
        weight_int: Quantized weights [8, 8, 3, 3]
    """

    device = x_pad_in.device

    # Reshape weights to [out_ch, in_ch, kij]
    W = weight_int.reshape(weight_int.size(0), weight_int.size(1), -1)
    print(f"Weight shape after reshape: {W.shape}")  # Should be [8, 8, 9]

    # Flatten input activations
    X = x_pad_in.reshape(x_pad_in.size(0), -1)
    print(f"Input shape after flatten: {X.shape}")  # Should be [8, 36]

    # Initialize partial sum memory [out_ch, spatial_pos, kij]
    psum_memory = torch.zeros(8, 36, 9, device=device)

    print("\nSimulation parameters:")
    print(f"- Total output channels: {W.size(0)}")
    print(f"- Input channels: {W.size(1)}")
    print(f"- Spatial positions (nij): {X.size(1)}")
    print(f"- Kernel elements (kij): {W.size(2)}")

    cycle_count = 0

    # Process each kernel element
    for kij in range(9):
        print(f"\n--- Processing kij = {kij} ---")

        # Get weight tile for this kernel element [out_ch, in_ch]
        w_tile_kij = W[:, :, kij]  # [8, 8]
        print(f"  Weight tile shape for kij={kij}: {w_tile_kij.shape}")

        # Process each spatial position
        for nij in range(X.size(1)):
            # Get input at this position and kernel element
            x_nij_kij = X[:, nij]  # [8]

            # Compute matrix multiplication: [out_ch] = [out_ch, in_ch] ×␣
↪[in_ch]
            # This represents the MAC array output (8 outputs for 8 output␣
↪channels)
            mac_result = torch.matmul(w_tile_kij, x_nij_kij)  # [8]

            # Display MAC array output in hex format
```

```
            if cycle_count < 30:  # Show first 30 cycles
                print(f"    Cycle {cycle_count}: kij={kij}, nij={nij}")
                print(f"      MAC Array Output (8 channels):")
                mac_output_line = "      "
                for idx, val in enumerate(mac_result):
                    int_val = int(round(val.item()))
                    # Convert to 16-bit two's complement hex
                    if int_val < 0:
                        hex_val = format(int_val & 0xFFFF, '04X')  # 16-bit␣
 ↪two's complement
                    else:
                        hex_val = format(int_val & 0xFFFF, '04X')
                    mac_output_line += f"och[{idx}]:{hex_val} "
                print(mac_output_line)

            # Store partial sums in memory
            psum_memory[:, nij, kij] = mac_result

            cycle_count += 1

    return psum_memory

# Alternative function with more compact display
def generate_debug_data_compact_mac_output(x_pad_in, weight_int):
    """
    Generate debug data with compact MAC array output display
    """

    device = x_pad_in.device

    # Reshape weights to [out_ch, in_ch, kij]
    W = weight_int.reshape(weight_int.size(0), weight_int.size(1), -1)

    # Flatten input activations
    X = x_pad_in.reshape(x_pad_in.size(0), -1)

    # Initialize partial sum memory [out_ch, spatial_pos, kij]
    psum_memory = torch.zeros(8, 36, 9, device=device)

    cycle_count = 0

    # Process each kernel element
    for kij in range(9):
        # Process each spatial position
        for nij in range(X.size(1)):
            # Get input at this position and kernel element
            x_nij_kij = X[:, nij]  # [8]
```

```
            # Compute matrix multiplication
            mac_result = torch.matmul(W[:, :, kij], x_nij_kij)  # [8]

            # Display MAC array output in hex format (compact)
            if cycle_count < 20:  # Show first 20 cycles
                print(f"Cycle {cycle_count:2d} | ", end="")
                for idx, val in enumerate(mac_result):
                    int_val = int(round(val.item()))
                    hex_val = format(int_val & 0xFFFF, '04X')
                    print(f"{hex_val} ", end="")
                print()  # New line

            # Store partial sums in memory
            psum_memory[:, nij, kij] = mac_result

            cycle_count += 1

    return psum_memory

# Function to save MAC outputs to file
def save_mac_outputs_to_file(x_pad_in, weight_int, filename='mac_outputs_4b4b.
 ↪txt'):
    """
    Save all MAC array outputs to a file
    """

    device = x_pad_in.device

    # Reshape weights to [out_ch, in_ch, kij]
    W = weight_int.reshape(weight_int.size(0), weight_int.size(1), -1)

    # Flatten input activations
    X = x_pad_in.reshape(x_pad_in.size(0), -1)

    with open(filename, 'w') as f:
        f.write("// MAC Array Outputs for 4b4b Systolic Array\n")
        f.write("// Format: Cycle | och0 | och1 | och2 | och3 | och4 | och5 |␣
 ↪och6 | och7\n")

        cycle_count = 0

        # Process each kernel element
        for kij in range(9):
            # Process each spatial position
            for nij in range(X.size(1)):
                # Get input at this position and kernel element
```

```
                x_nij_kij = X[:, nij]  # [8]

                # Compute matrix multiplication
                mac_result = torch.matmul(W[:, :, kij], x_nij_kij)  # [8]

                # Write to file
                line = f"{cycle_count:3d} "
                for val in mac_result:
                    int_val = int(round(val.item()))
                    hex_val = format(int_val & 0xFFFF, '04X')
                    line += f"{hex_val} "
                f.write(line.strip() + "\n")

                cycle_count += 1

    print(f"MAC outputs saved to {filename}")

# Usage example with your existing data:
# Assuming x_pad_in and weight_int are available from your context

# Method 1: Verbose display
print("=== Method 1: Verbose MAC Output Display ===")
psum_memory = generate_debug_data_4b4b_with_mac_output(x_pad_in, weight_int)

print("\n" + "="*50)
print("=== Method 2: Compact MAC Output Display ===")
# Method 2: Compact display
psum_memory_compact = generate_debug_data_compact_mac_output(x_pad_in,␣
 ↪weight_int)

print("\n" + "="*50)
# Method 3: Save to file
print("=== Method 3: Saving MAC Outputs to File ===")
save_mac_outputs_to_file(x_pad_in, weight_int)
```